

Quinta Lista de Engenharia de Software RT

Capítulo 5 — Modelagem de Sistemas (UML)

Ian Sommerville — *Software Engineering*, 10ª ed. + Exercícios IAS — Universität Stuttgart
(Exercises 2, 3 e 8)

Aluno: Matheus Serrão Uchoa

Esta lista é dividida em quatro partes: a Parte I cobre as questões do Capítulo 5 de Sommerville (modelagem de sistemas); as Partes II, III e IV correspondem, respectivamente, aos *Exercises 2, 3 e 8* do material do *Institute of Industrial Automation and Software Engineering* (IAS) da Universität Stuttgart, sobre modelagem orientada a objetos em UML.

Sumário

1	Parte I — Capítulo 5 (Sommerville): Modelagem de Sistemas	1
2	Parte II — Exercise 2 (Stuttgart): Introdução à UML	9
3	Parte III — Exercise 3 (Stuttgart): Introdução à UML (2)	13
4	Parte IV — Exercise 8 (Stuttgart): Análise de Casos de Uso	15

1 Parte I — Capítulo 5 (Sommerville): Modelagem de Sistemas

Questão 1 — Importância de modelar o contexto

Enunciado

Explique por que é importante modelar o contexto de um sistema que está sendo desenvolvido. Dê dois exemplos de possíveis erros que podem ocorrer, caso os engenheiros de software não entendam o contexto do sistema.

Resposta

Modelar o contexto significa estabelecer explicitamente a **fronteira do sistema** — o que pertence ao software a ser construído e o que pertence ao seu ambiente (outros sistemas, pessoas e organizações com os quais ele interage). Isso é importante porque:

- **Define o escopo:** deixa claro quais funcionalidades são responsabilidade do novo sistema e quais já são fornecidas pelo ambiente, evitando duplicação ou lacunas.
- **Identifica interfaces:** revela todos os sistemas externos com os quais haverá troca de dados, determinando interfaces que precisam ser especificadas e testadas.
- **Expõe dependências sociais e organizacionais:** a fronteira não é apenas técnica; decisões sobre o que automatizar afetam processos de trabalho e pessoas.

Exemplos de erros decorrentes de não entender o contexto:

1. **Funcionalidade omitida em uma interface externa:** ao desenvolver o sistema de prontuário de uma clínica (MHC-PMS) sem perceber que ele depende do *sistema de registro de pacientes* do hospital, a equipe deixa de implementar a sincronização

de cadastros. Resultado: dados inconsistentes (um paciente alterado em um sistema não reflete no outro).

2. **Fronteira posicionada erroneamente:** supor que a validação de cobrança seria feita pelo sistema de faturamento já existente, quando na verdade esse sistema não a faz, leva a transações aceitas indevidamente — uma função crítica fica "no vácuo" entre os dois sistemas.

Questão 2 — Modelo de sistema existente; completude e correção

Enunciado

Como você poderia usar um modelo de um sistema que já existe? Explique por que nem sempre é necessário que um modelo de sistema seja completo e correto. O mesmo seria verdadeiro caso você estivesse desenvolvendo um modelo de um novo sistema?

Resposta

Usos de um modelo de um sistema existente:

- **Compreensão e comunicação:** entender como o sistema atual funciona antes de mantê-lo, estendê-lo ou substituí-lo.
- **Documentação e reengenharia:** servir de base para discutir melhorias, identificar componentes reutilizáveis e planejar a migração.
- **Treinamento e suporte** de novos integrantes da equipe.

Por que não precisa ser completo e correto: um modelo é uma *abstração* cujo propósito é facilitar a comunicação sobre aspectos *relevantes para uma decisão específica*. Incluir todos os detalhes o tornaria tão complexo quanto o próprio sistema e ilegível. Para um sistema existente, modelamos seletivamente apenas as partes de interesse (por exemplo, o subsistema que será alterado), omitindo deliberadamente o resto sem prejuízo.

Para um novo sistema, o mesmo seria verdadeiro? Parcialmente. O modelo continua sendo uma abstração e *não precisa ser completo* em todos os estágios — em desenvolvimento incremental ele evolui. Porém, como agora o modelo também funciona como *especificação que orienta a construção*, a tolerância à **incorreção é muito menor**: um erro no modelo do novo sistema tende a se propagar para a implementação como defeito. Em resumo: completude não é exigida em nenhum dos casos, mas a correção é mais crítica no modelo de um sistema novo do que no modelo de um sistema já existente (que apenas descreve algo já pronto).

Questão 3 — Diagrama de atividades: planejamento de festas

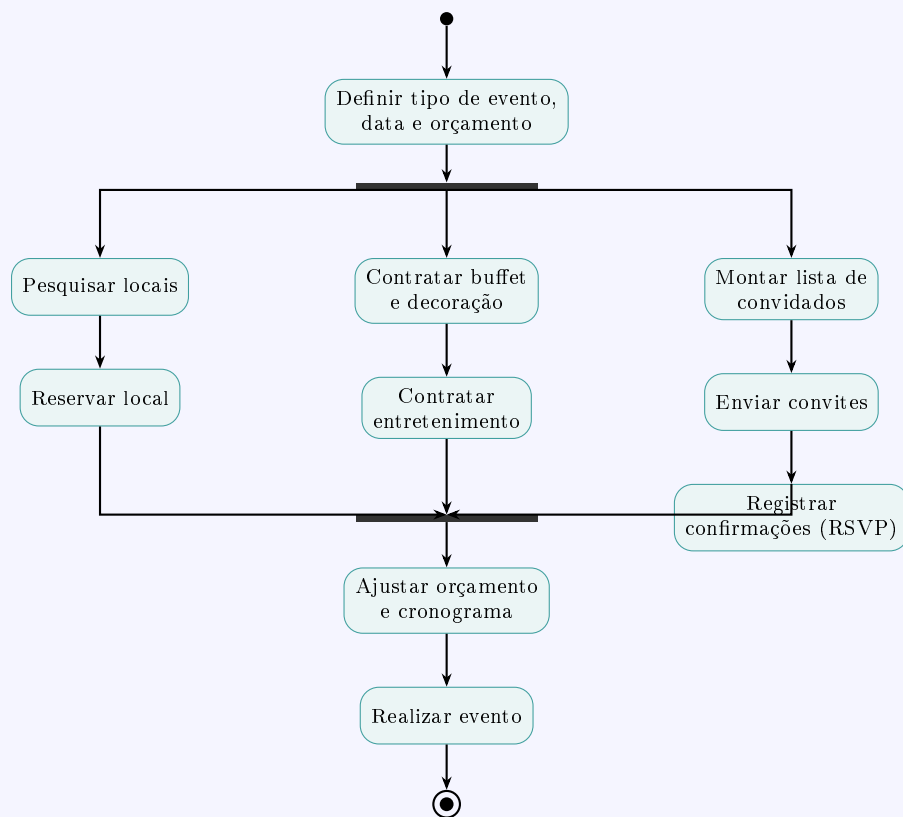
Enunciado

Modele, com um diagrama de atividades, o contexto do processo de um sistema que mostre as atividades envolvidas no planejamento de uma festa (reserva de local, organização dos convites etc.) e os elementos do sistema usados em cada etapa.

Resposta

O diagrama abaixo mostra o fluxo de atividades. Após a definição inicial, três fluxos correm em **paralelo** (barra de *fork*): reserva de local, contratação de serviços e gestão de convidados. Ao lado de cada atividade indica-se, em *itálico*, o **elemento/subsistema** de

apoio utilizado.



Elementos do sistema utilizados em cada etapa:

Atividade	Elemento / subsistema de apoio
Pesquisar locais	Catálogo de locais
Reservar local	Módulo de reservas / pagamento
Contratar buffet, decoração e entretenimento	Cadastro de fornecedores
Enviar convites	Módulo de e-mail / mensagens
Registrar confirmações (RSVP)	Banco de dados de convidados
Ajustar orçamento e cronograma	Planejador / agenda

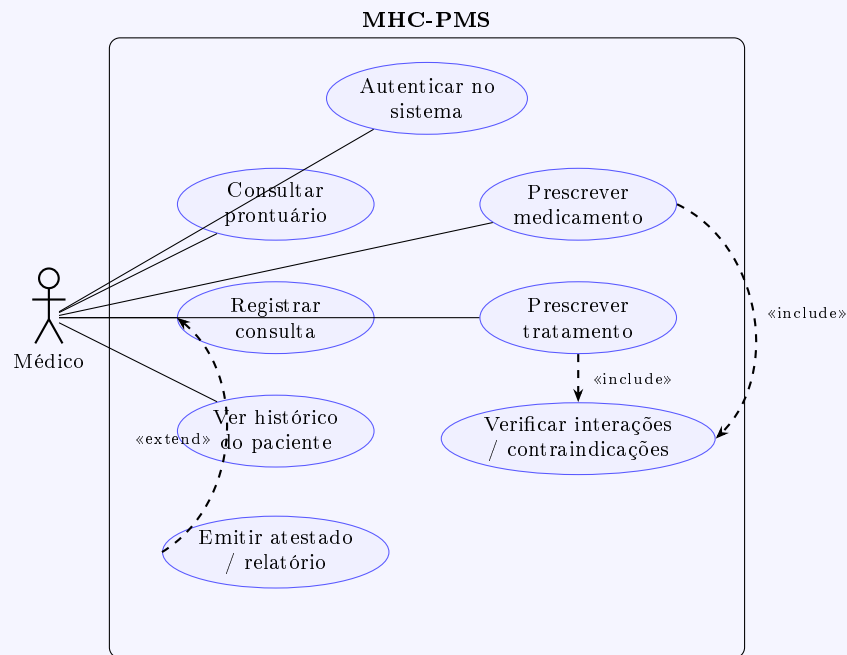
Questão 4 — Casos de uso do MHC-PMS (médico)

Enunciado

Para o MHC-PMS, proponha um conjunto de casos de uso que ilustre as interações entre um médico (que atende pacientes e prescreve medicamentos e tratamentos) e o MHC-PMS.

Resposta

O médico (ator) interage com o sistema sempre após autenticação. As prescrições **incluem** («include») a verificação de contraindicações/interações, e o registro de consulta pode ser **estendido** para emissão de atestado.



Descrição resumida dos casos de uso:

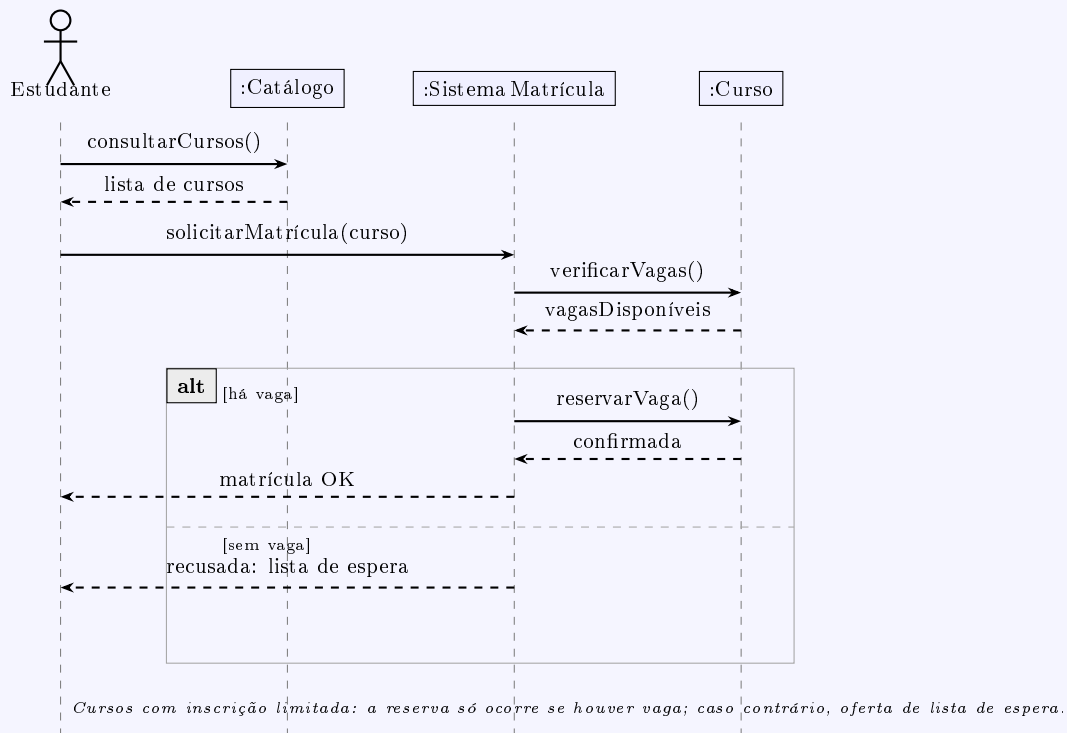
- **Autenticar:** médico fornece credenciais; o sistema libera o acesso conforme seu papel.
- **Consultar prontuário:** localizar paciente e abrir seus dados.
- **Registrar consulta:** anotar sintomas, diagnóstico e evolução.
- **Prescrever medicamento / tratamento:** indicar fármacos ou terapias; o sistema verifica automaticamente interações e alergias.
- **Ver histórico:** visualizar consultas, prescrições e exames anteriores.
- **Emitir atestado/relatório:** gerar documentos a partir da consulta.

Questão 5 — Diagrama de sequência: matrícula em curso

Enunciado

Desenvolva um diagrama de sequência mostrando as interações quando um estudante se registra para um curso. Os cursos podem ter inscrição limitada, então o processo deve incluir verificação de vagas. Suponha que o estudante consulte um catálogo eletrônico de cursos.

Resposta

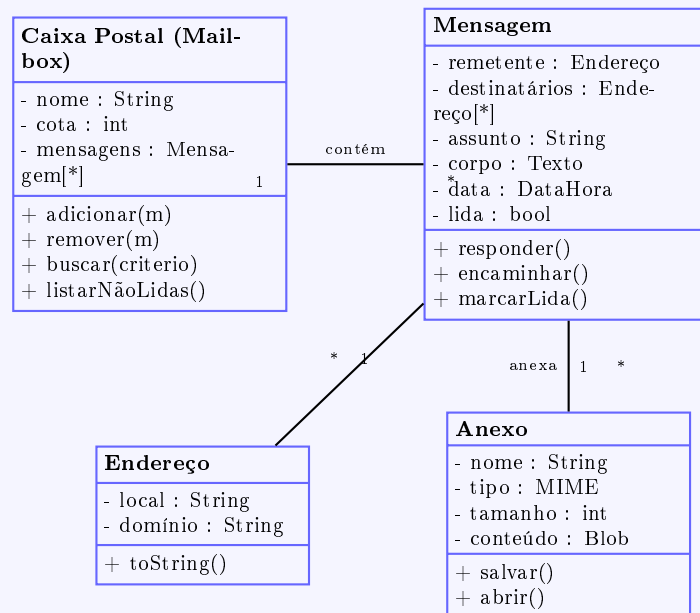


Questão 6 — Classes: caixa postal e mensagem de e-mail

Enunciado

Modele as classes de objetos que poderiam ser usadas na implementação de um sistema de e-mail para representar uma caixa postal e uma mensagem de correio eletrônico.

Resposta



Notas de projeto: a *Caixa Postal* agrega muitas *Mensagens* (composição: ao apagar a

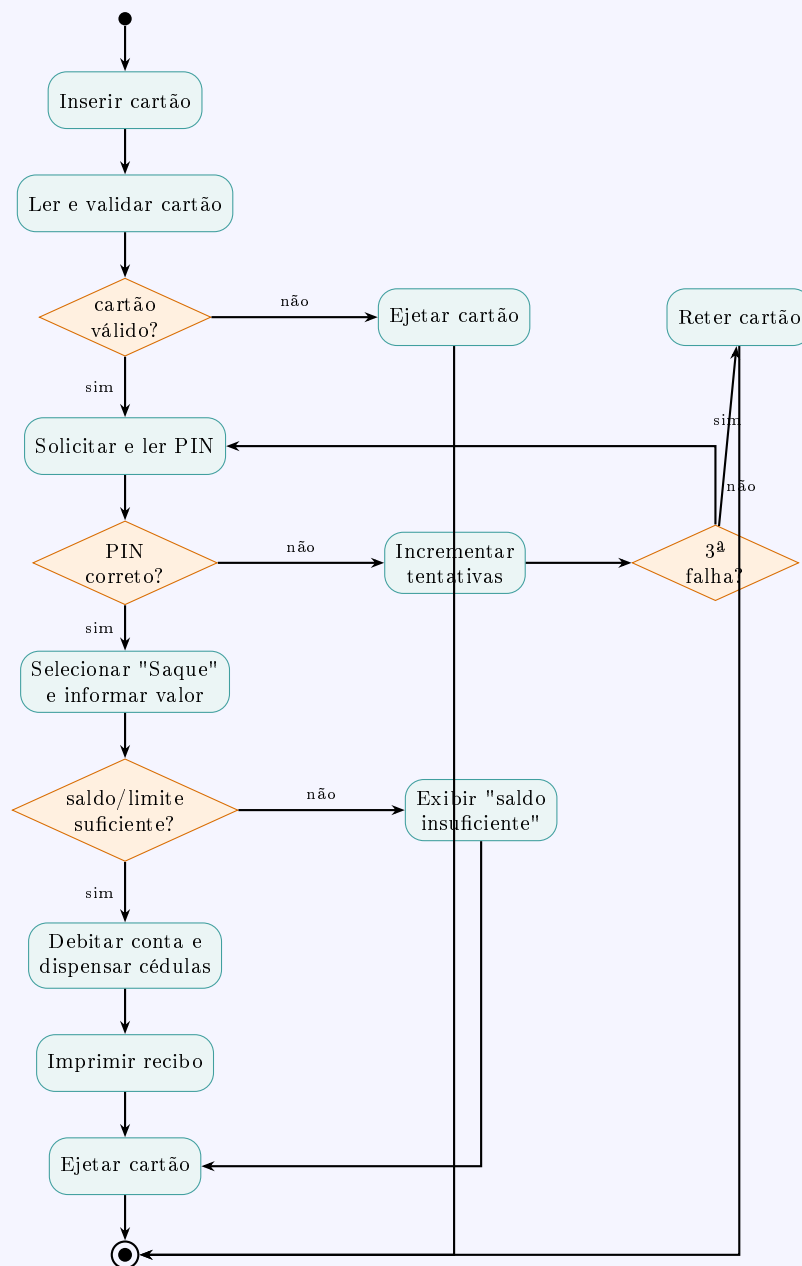
caixa, as mensagens vão junto); cada *Mensagem* pode ter vários *Anexos* e referencia *Endereços* (remetente/destinatários), modelados como classe própria para reúso e validação.

Questão 7 — Diagrama de atividades: saque em caixa eletrônico

Enunciado

Desenhe um diagrama de atividades que modele o processamento de dados envolvido quando um cliente retira dinheiro de um caixa eletrônico (ATM).

Resposta

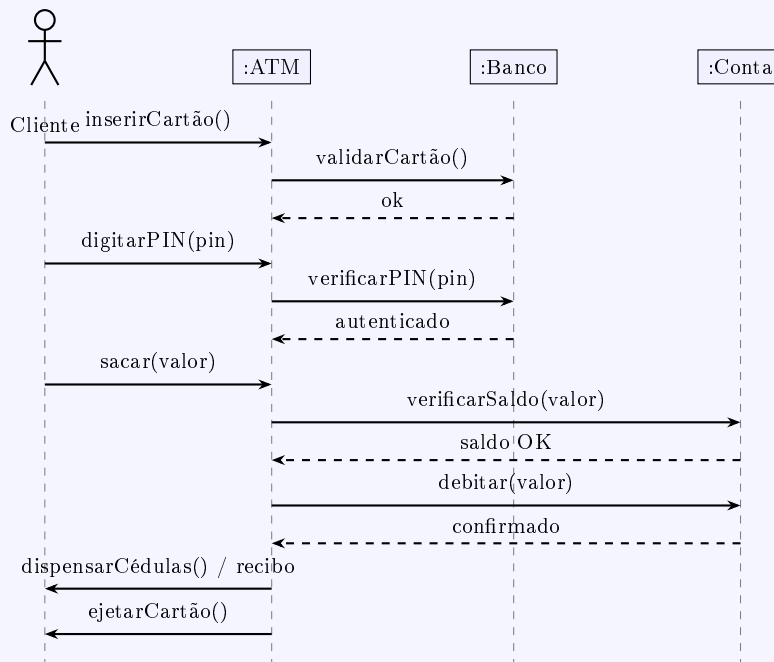


Questão 8 — Diagrama de sequência do ATM; por que ambos

Enunciado

Desenhe um diagrama de sequência para o mesmo sistema (ATM). Explique por que você pode querer desenvolver tanto diagramas de atividades quanto de sequência ao modelar o comportamento de um sistema.

Resposta



Por que desenvolver os dois tipos de diagrama?

- O **diagrama de atividades** é orientado ao *fluxo de controle/processo*: enfatiza a ordem das ações, decisões, laços e paralelismo, *sem* se comprometer com quais objetos executam o quê. É ideal para entender a lógica do negócio e validar regras com usuários.
- O **diagrama de sequência** é orientado à *interação entre objetos* ao longo do tempo: mostra *quem* envia *qual* mensagem para *quem*, revelando responsabilidades e interfaces. É ideal para o projeto detalhado e para distribuir comportamento entre as classes.

São visões **complementares**: a de atividades responde "*o que acontece e em que ordem*"; a de sequência responde "*como os objetos colaboram para realizar esse fluxo*". Usar ambas reduz ambiguidades e ajuda a verificar se o projeto de objetos realmente implementa o processo desejado.

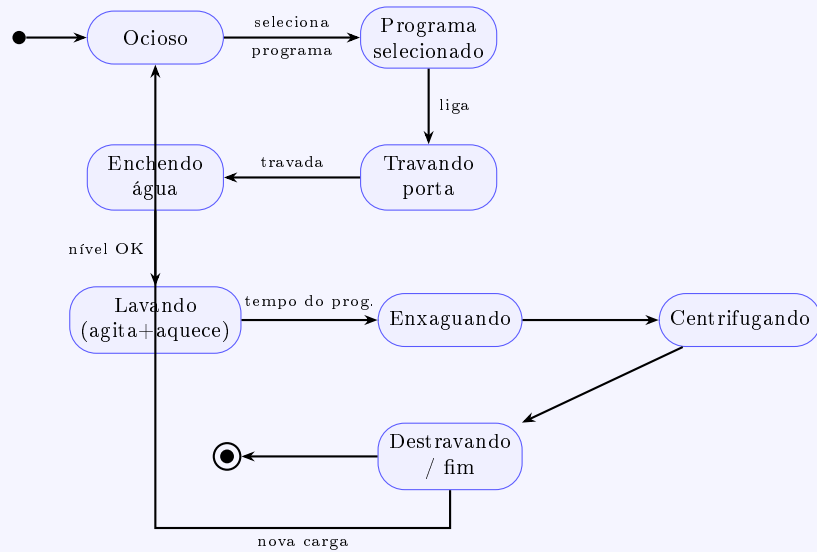
Questão 9 — Diagramas de estado (3 sistemas)

Enunciado

Desenhe diagramas de estado do software de controle para: (a) uma máquina de lavar automática com programas diferentes; (b) um *DVD player*; (c) um sistema de atendimento telefônico (secretária eletrônica) que grava mensagens e exibe o número de mensagens em um LED, permitindo escutar mensagens à distância por tons (DTMF).

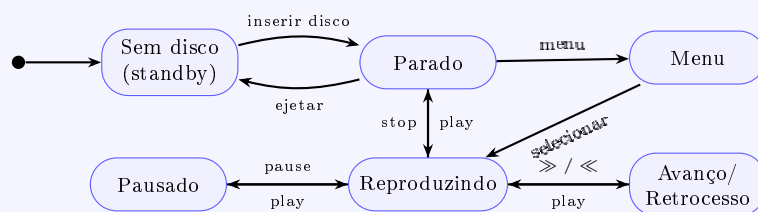
Resposta

(a) Máquina de lavar automática

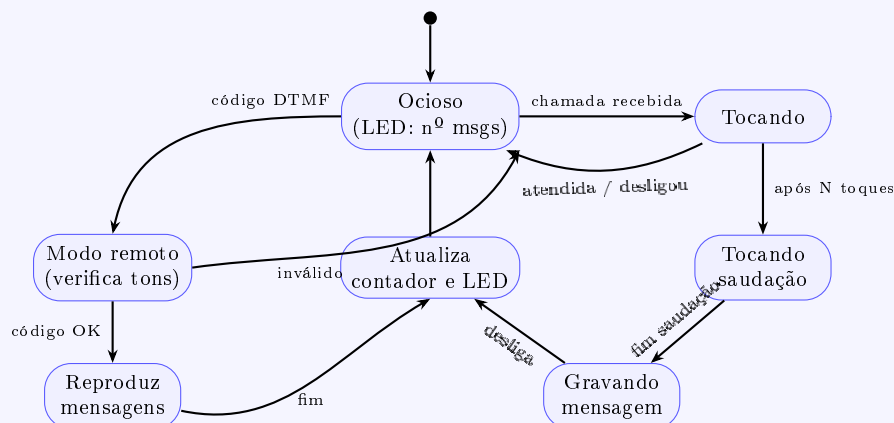


O programa selecionado (algodão, sintéticos, delicados...) parametriza temperatura, duração e rotação; o comportamento de estados é o mesmo, mudam os valores (entry actions de cada estado configuram os parâmetros do programa).

(b) DVD player



(c) Secretária eletrônica (atendimento telefônico)



O cliente disca de qualquer lugar, digita a sequência de tons (DTMF) e, autenticado, ouve as mensagens; o LED sempre reflete o número de mensagens aceitas.

Questão 10 — Fatores para adotar Engenharia Dirigida a Modelos

Enunciado

Como gerente, que fatores você deve considerar ao decidir se introduz ou não a engenharia dirigida a modelos (MDE) no desenvolvimento de um novo sistema?

Resposta

- **Maturidade de ferramentas e padrões:** existem ferramentas MDA/MDE estáveis para o domínio? Geradores de código confiáveis? A adoção exige ferramentas que ainda variam muito em qualidade.
- **Adequação do domínio:** MDE rende mais em domínios estáveis e bem compreendidos, com boas linguagens específicas de domínio (DSL) e abstrações independentes de plataforma (PIM).
- **Longevidade do sistema:** sistemas de vida longa, que sobrevivem a várias mudanças de plataforma, se beneficiam — o modelo (PIM) é portátil; gera-se novo PSM quando a tecnologia muda.
- **Necessidade de portabilidade** entre múltiplas plataformas-alvo.
- **Competências e custo de treinamento:** curva de aprendizado da equipe em metamodelagem, transformações e DSLs.
- **Desempenho:** código gerado automaticamente pode ser menos otimizado que código artesanal — crítico em sistemas de tempo real.
- **Integração com o processo existente:** controle de versão de modelos, depuração no nível de modelo, integração com a cadeia de ferramentas e cultura da equipe.
- **Retorno do investimento (ROI):** alto custo inicial *vs.* ganhos de produtividade e manutenção no longo prazo. Sommerville observa que a adoção da MDE ainda é limitada justamente por esse trade-off.

2 Parte II — Exercise 2 (Stuttgart): Introdução à UML

Observação

Resumo dos conceitos (slides do Exercise 2). *Objeto:* entidade concreta com estado, comportamento e identidade. *Classe:* abstração que descreve estrutura (atributos), comportamento (operações) e relacionamentos (associações) de um conjunto de objetos. *Associação:* ligação entre objetos, com multiplicidade em cada extremidade. *Agregação:* relação todo–parte ("é parte de"). *Herança/generalização:* relação entre classe geral (base) e especializada (derivada), que herda e estende a base.

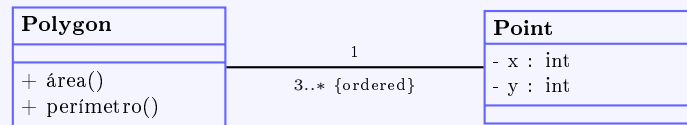
Questão 2.1 — Associações entre objetos (polígono)

Enunciado

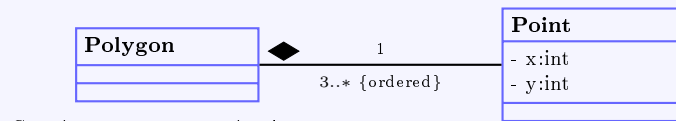
(a) Derive um diagrama de classes a partir do diagrama de objetos do polígono (sem multiplicidades ainda); cada ponto tem coordenadas X e Y — como expressar que os pontos estão em ordem? (b) Qual o número mínimo de pontos para um polígono? Acrescente multiplicidades para os casos: (1) cada ponto pertence a exatamente um polígono; (2) um ponto pode ser compartilhado por vários polígonos. (c) Para dois triângulos que compartilham um lado, desenhe um diagrama de objetos para cada caso de (b).

Resposta

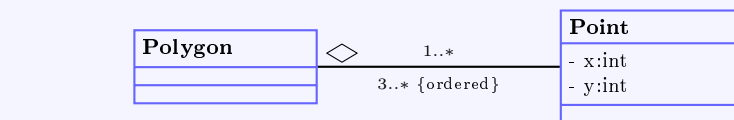
(a) **Diagrama de classes.** Identificam-se duas classes, *Polygon* e *Point*. A **ordem** dos pontos é expressa pela restrição **{ordered}** na extremidade da associação (ou, equivalentemente, por uma associação qualificada/indexada por um número de sequência).



(b) **Mínimo de pontos:** um polígono precisa de **pelo menos 3** pontos (vértices). Multiplicidades para cada caso:

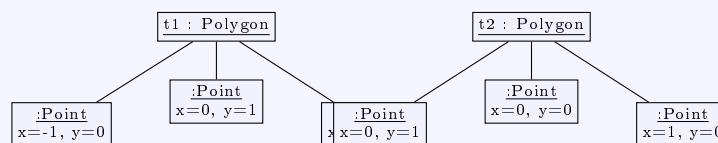


Caso 1: ponto pertence a 1 polígono

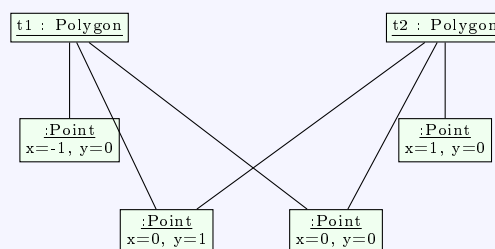


Caso 2: ponto compartilhado por vários polígonos

(c) **Diagramas de objetos — dois triângulos com um lado comum** (lado compartilhado entre os vértices (0,0) e (0,1)):



Caso 1: 6 objetos Point (os 2 vértices do lado comum são duplicados)



Caso 2: 4 objetos Point (os 2 vértices do lado comum são compartilhados)

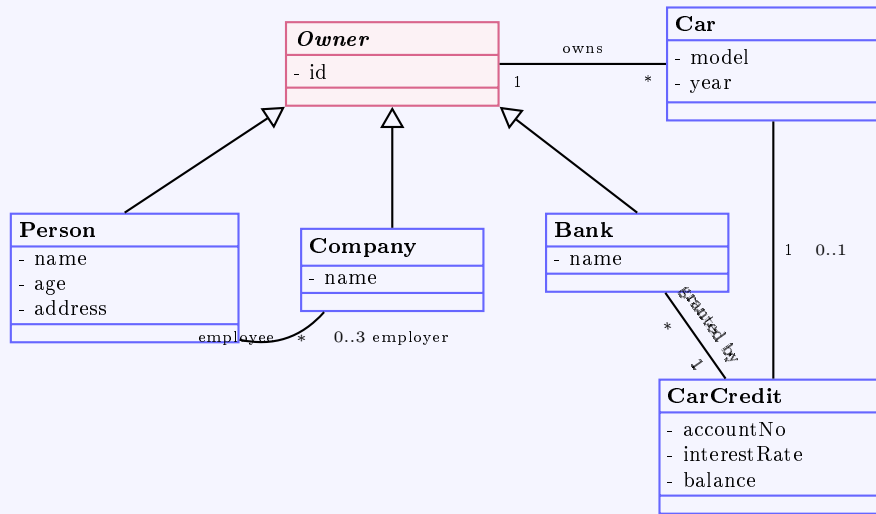
Questão 2.2 — Associações entre classes (gestão de carros)

Enunciado

No diagrama original, várias classes contêm atributos que na verdade são ponteiros para outros objetos. Redesenhe o diagrama de classes substituindo os ponteiros por associações, introduzindo generalizações onde útil e determinando as multiplicidades.

Resposta

Os atributos-ponteiro (*Employer ID*, *Owner ID*, *Type of owner*, *Bank ID* etc.) são eliminados. Como um carro pode ser de uma *Person*, *Company* ou *Bank*, introduz-se uma superclasse abstrata *Owner* (proprietário), generalizando as três.



Multiplicidades: uma *Person* pode ter de 0 a 3 *Company* como empregador; um *Car* tem exatamente 1 *Owner* (que pode possuir vários carros); a compra de um *Car* pode ter 0 ou 1 *CarCredit*; cada *CarCredit* é concedido por exatamente 1 *Bank*.

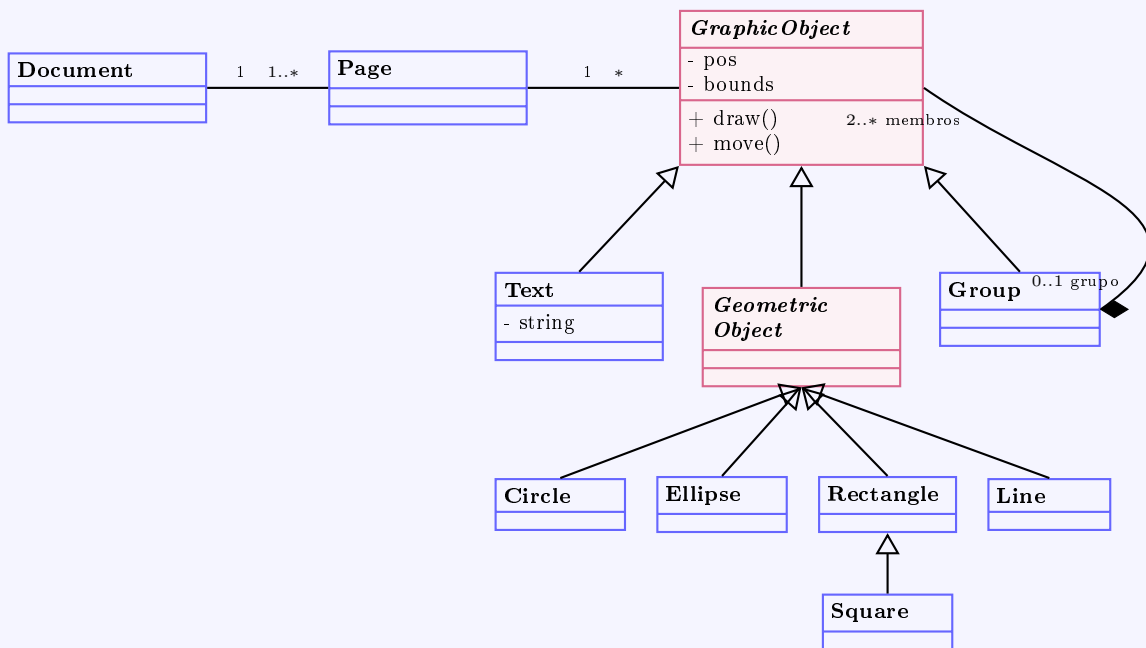
Questão 2.3 — Diagrama de classes de um editor gráfico

Enunciado

Desenhe um diagrama de classes para um editor gráfico que suporte agrupamento. Um documento tem várias páginas; cada página contém objetos gráficos: texto, objetos geométricos e grupos. Um grupo é um conjunto de objetos gráficos (pode conter outros grupos), com no mínimo dois objetos; um objeto gráfico é membro direto de no máximo um grupo.

Resposta

A solução é o padrão **Composite**: *GraphicObject* é a superclasse abstrata; *Group* é simultaneamente um *GraphicObject* e um contêiner de *GraphicObjects*.



Multiplicidades-chave: *Group* agrega **2..*** *GraphicObjects*; cada *GraphicObject* é membro direto de **0..1** grupo. Como *Group* herda de *GraphicObject*, grupos podem conter grupos (recursão), satisfazendo o requisito de aninhamento.

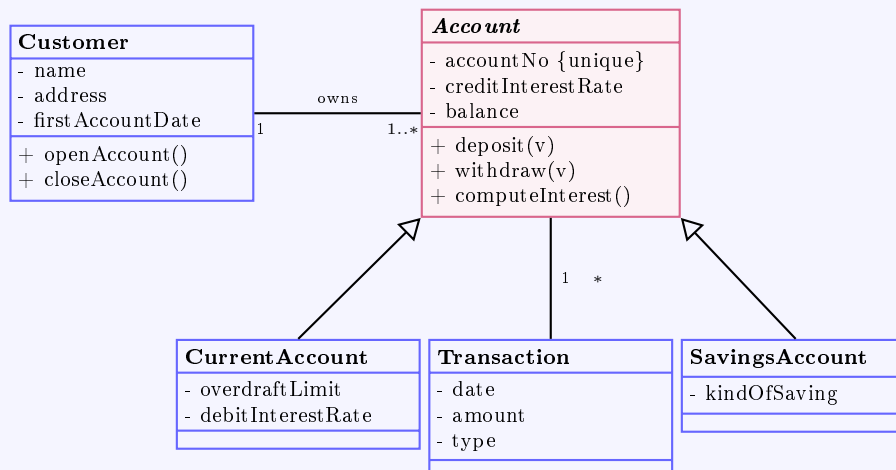
Questão 2.4 — Diagrama de classes de um software bancário

Enunciado

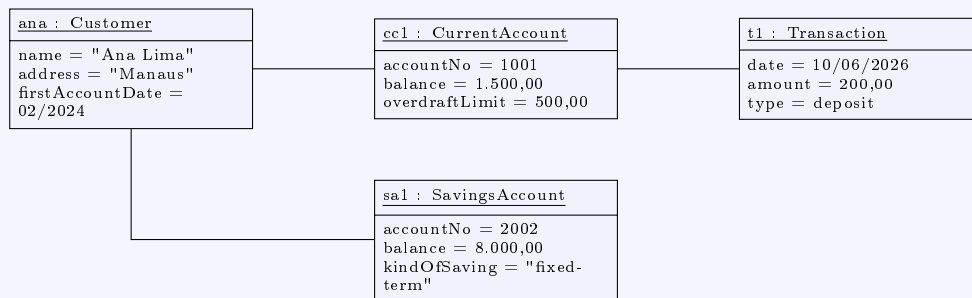
(a) Identifique classes, atributos, operações, associações e estruturas de herança na descrição do sistema bancário e desenhe um diagrama de classes. (b) Crie um diagrama de objetos com dados de exemplo.

Resposta

(a) **Diagrama de classes.** Um *Customer* surge ao abrir a primeira *Account* e deixa de existir ao encerrar a última. Distinguem-se *CurrentAccount* (com limite de cheque especial e juros de débito) e *SavingsAccount* (com tipo de poupança). Cada *Transaction* (depósito/saque/juros) registra data e valor.



(b) Diagrama de objetos (dados de exemplo):



3 Parte III — Exercise 3 (Stuttgart): Introdução à UML (2)

Questão 3.1 — Identificação de classes (biblioteca universitária)

Enunciado

(a) Os requisitos estão completos? Que outras perguntas deveriam ser feitas? (b) Identifique as classes candidatas pelos substantivos. (c) Descarte os substantivos pouco adequados a classe, justificando. (d) Outras sugestões de solução?

Resposta

(a) **Requisitos completos? Não.** Perguntas adicionais:

- Como funciona a fila de *reservas* (FIFO)? Há notificação quando o item fica disponível?
- Quais dados do *membro* são armazenados? Há taxa de associação?
- Há *multas* por atraso? Qual o valor e a política?
- Funcionários da biblioteca também podem ser membros? Que *permissões* cada papel possui?
- Como se tratam itens *perdidos ou danificados*?
- Por qual meio os *lembretes* são enviados (e-mail, correio)?
- Quais *critérios de busca* (título, autor, tópico, ISBN)?
- Como é rastreado o estado de *cada cópia* (disponível, emprestada)?
- Quantas vezes um empréstimo pode ser *prorrogado*?

(b) **Classes candidatas (substantivos):** Library, Book, Magazine, Copy, Member, Employee, User, Borrowing/Loan, Reminder, Reservation, Topic, Author, Catalogue/Card, System, Year, Unit, Person, Time span.

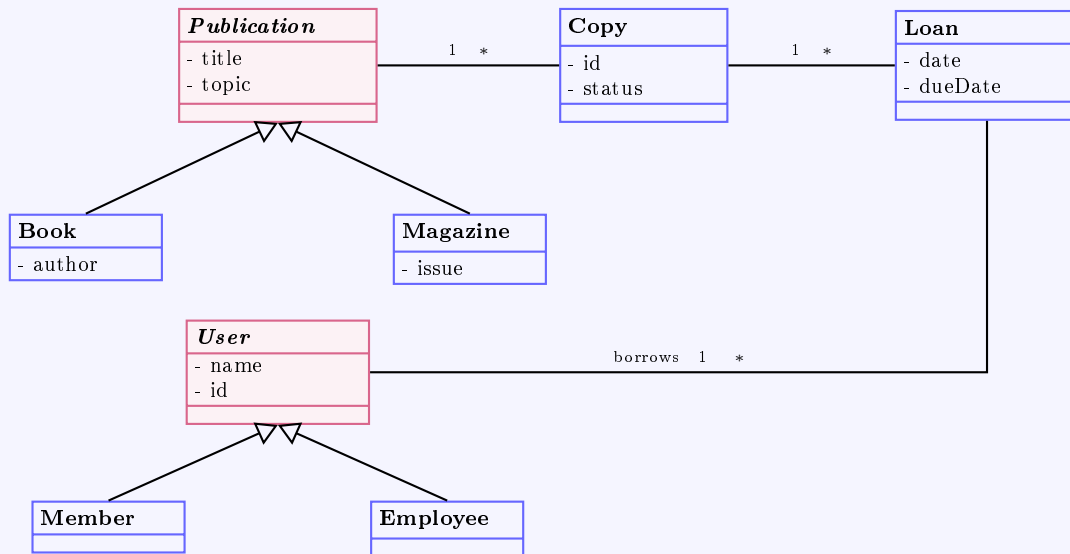
(c) **Descarte (com justificativa):**

- **System, Library** — representam o próprio sistema/contexto, não objetos dentro dele.
- **Year, Time span** — valores/atributos (ano de edição, prazo).
- **Unit** — sinônimo genérico de *Copy*/item; redundante.
- **Catalogue / Card** — artefato da interface ou do sistema legado; corresponde à *função de busca*, não a uma classe de domínio.
- **Topic, Author** — atributos de *Book* (ou pequenas classes de valor, se houver busca estruturada por autor).
- **User** — termo guarda-chuva; melhor modelar como superclasse de *Member* e *Employee*.

Classes mantidas: Publication (super) → Book, Magazine; Copy; Person/User (super)

→ Member, Employee; Loan; Reservation; Reminder.

(d) Sugestão de solução (esboço):



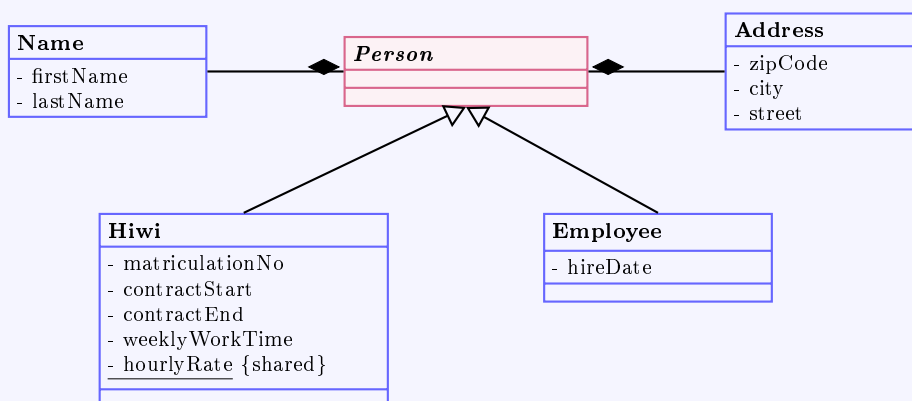
Questão 3.2 — Identificar classes e atributos (funcionários)

Enunciado

Identifique classes e atributos para a administração de *hiwis* (estudantes contratados) e funcionários normais de uma universidade e represente-os em um diagrama. Há classes elementares?

Resposta

Person é a superclasse abstrata. *Name* e *Address* são **classes elementares** (de valor), agregadas por composição. O atributo *hourlyRate* é **de classe** (sublinhado), pois é o mesmo para todo *hiwi*.



Sim, existem classes elementares: *Name* e *Address*, que apenas estruturam dados e não têm comportamento próprio relevante.

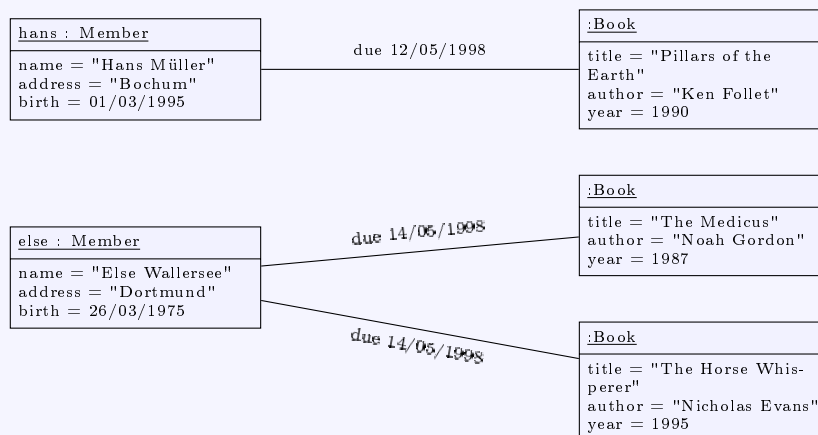
Questão 3.3 — Diagrama de objetos e de classes (livros)

Enunciado

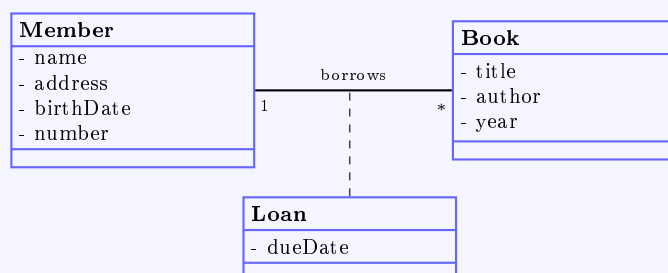
- (a) Identifique objetos e associações e represente-os em um diagrama de objetos. (b) Identifique as classes e associações e represente-as em um diagrama de classes.

Resposta

(a) Diagrama de objetos:



(b) Diagrama de classes (com *Loan* como classe de associação):



4 Parte IV — Exercise 8 (Stuttgart): Análise de Casos de Uso

Observação

Cenário (máquina de venda de bilhetes). A *Transport Authority Undelayed* encomendou o software de controle das novas máquinas de bilhetes. Elementos: *Destination Key Panel* (código de 3 dígitos do destino), *Ticket Key Panel* (tipo: ida única / múltiplas viagens, normal / primeira classe), *Continue/Cancel Key Panel*, *Display*, *Card Drive* (GeldKarte), *Coin/Note Slots* e *Printer*. Assume-se pagamento exato (sem troco). A manutenção é feita por técnicos de serviço.

Questão 8.1 — Modelo de casos de uso

Enunciado

(a) Nomeie o sistema e os atores externos. (b) Liste os casos de uso, distinguindo os fechados (independentes) dos parciais (contidos em outros). (c) Especifique-os com os *templates*. (d) Apresente o diagrama de casos de uso.

Resposta

(a) **Sistema:** software de controle da *Máquina de Bilhetes* (*Ticket Machine*). **Atores externos:** *Passageiro* (*Passenger*) e *Técnico de Serviço* (*Service Technician*).

(b) **Casos de uso:**

- **Fechados (independentes):** *Comprar bilhete(s)* e *Manutenção da máquina*.
- **Parciais (contidos):** *Selecionar destino e tipo de bilhete* e *Pagar tarifa* — ambos incluídos em *Comprar bilhete(s)*.

(c) **Especificação (templates preenchidos):**

Use Case: Comprar bilhete(s) (*Categoria: Primary; Ator: Passageiro*)

Objetivo	Adquirir um ou vários bilhetes para um ou vários destinos.
Pré-condição	Máquina pronta; display mostra " <i>Selecione um destino</i> ".
Evento gatilho	Passageiro seleciona um destino.
Pós-cond. sucesso	Display volta a mostrar " <i>Selecione um destino</i> " (próximo passageiro).
Pós-cond. falha	Display mostra mensagem de erro.
Descrição	1) Selecionar destino e tipo ($\Rightarrow$ UC <i>Selecionar destino e tipo</i>); 2) Pagar tarifa ($\Rightarrow$ UC <i>Pagar tarifa</i>); 3) Imprimir bilhete e exibir " <i>Retire seu(s) bilhete(s)</i> "; 4) Após retirada, exibir " <i>Selecione um destino</i> ".
Extensões Alternativas	1a) Tecla <i>continuar</i> permite selecionar outro bilhete. 2a) Tecla <i>cancelar</i> aborta o processo.

Use Case: Selecionar destino e tipo de bilhete (*Primary; Passageiro*)

Objetivo	Selecionar destino e tipo para um bilhete.
Pré-condição	Display mostra " <i>Selecione um destino</i> ".
Evento gatilho	Passageiro seleciona um destino.
Pós-cond. sucesso	Display mostra os bilhetes selecionados e a tarifa total.
Descrição	1) Informar código do destino; exibir " <i>Ida única para XYZ</i> " e a tarifa.
Extensões	1a) Mudar para <i>múltiplas viagens</i> ; 1b) mudar para <i>primeira classe</i> (atualiza texto e tarifa).

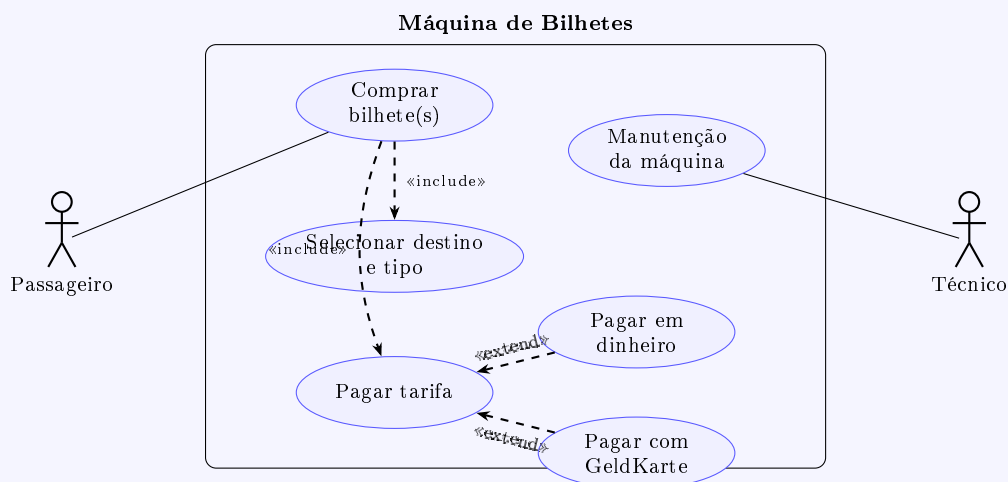
Use Case: Pagar tarifa (*Primary; Passageiro*)

Objetivo	Pagar a tarifa total dos bilhetes selecionados.
Pré-condição	Display mostra bilhetes selecionados e a tarifa total.
Evento gatilho	Passageiro insere a GeldKarte ou dinheiro.
Pós-cond. sucesso	Display mostra " <i>Seu(s) bilhete(s) está(ão) sendo impresso(s)</i> ".
Pós-cond. falha	Display indica crédito insuficiente / erro.
Descrição	1) Debitar a tarifa da GeldKarte se houver crédito; ejetar GeldKarte.
Alternativas	1a) Inserir dinheiro até atingir a tarifa (display atualiza o restante); 1b) crédito insuficiente é sinalizado.

Use Case: Manutenção da máquina (Secondary; Técnico de Serviço)

Objetivo	Manter a máquina pronta para uso.
Pré-condição	Intervalo de manutenção vencido ou display com erro.
Evento gatilho	Técnico abre a máquina.
Pós-cond. sucesso	Máquina pronta; display mostra "Selecione um destino".
Descrição	1) Executar manutenção/reparo; 2) autoteste dos componentes; se OK, exibir "Selecione um destino".
Alternativas	2a) Falha no autoteste ⇒ mensagem de erro.

(d) Diagrama de casos de uso:



Questão 8.2 — Identificação de objetos

Enunciado

Identifique os objetos relevantes no software de controle e dê uma breve descrição da finalidade de cada um.

Resposta

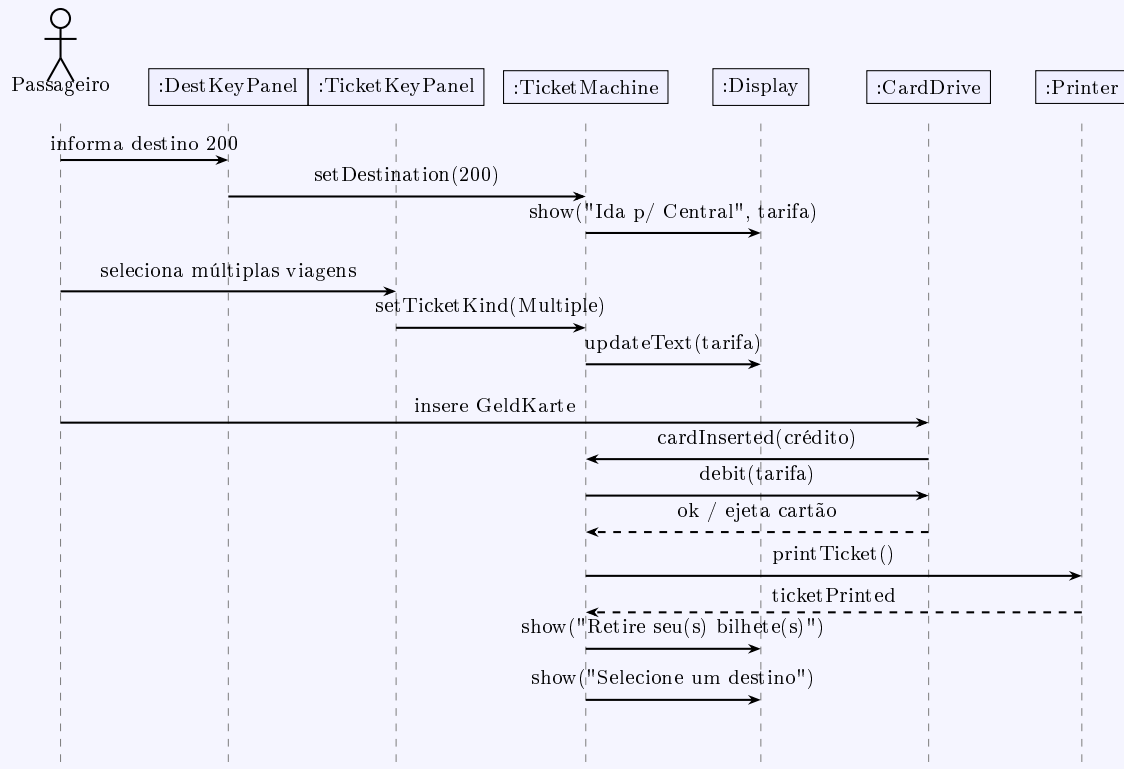
Objeto	Finalidade
Destination Key Panel	Permite a entrada do código do destino.
Ticket Key Panel	Permite selecionar o tipo de bilhete.
Cancel/Continue Key Panel	Permite cancelar o processo ou selecionar mais bilhetes.
Display	Fornece informações e orienta o usuário.
Card Drive	Lê o crédito da GeldKarte, debita a tarifa e ejeta o cartão.
Coin Slot / Note Slot	Soma o dinheiro inserido; sinaliza quando a tarifa foi paga.
Printer	Imprime e ejeta o bilhete; sinaliza quando foi retirado.
Ticket Machine	Nó central de controle; coordena os demais objetos e executa o que os periféricos não fazem.

Questão 8.3 — Modelo de análise dinâmica (sequência)

Enunciado

Crie um diagrama de sequência para o caso de uso *Comprar bilhete(s)* no cenário: destino 200 (Central Station); múltiplas viagens; pagamento com GeldKarte; há crédito suficiente.

Resposta



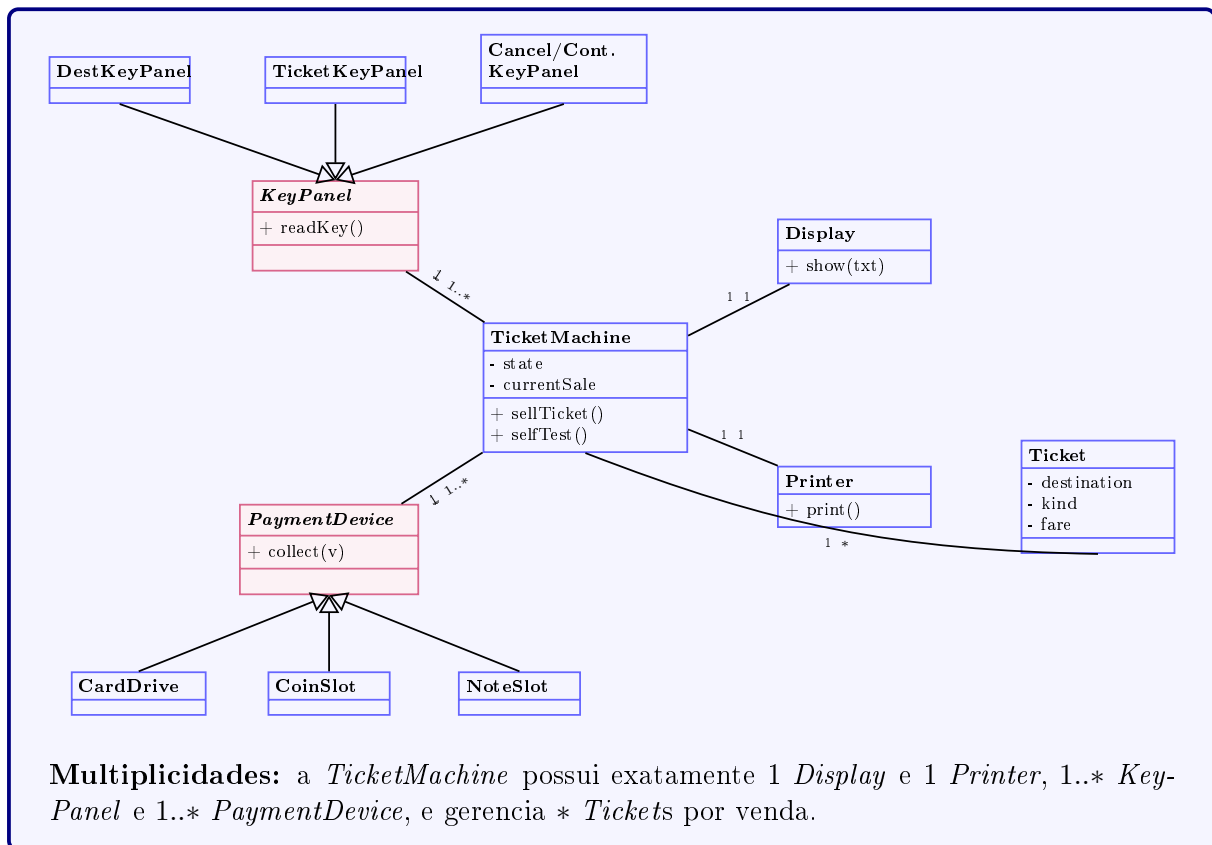
Questão 8.4 — Modelo de análise estática (classes)

Enunciado

Crie um modelo de análise estática (diagrama de classes) do software de controle. Cada componente de hardware é representado por uma classe de controle. Introduza herança de forma razoável e especifique as multiplicidades.

Resposta

A *TicketMachine* (controladora) associa-se a cada classe de controle de hardware. Introduzem-se as superclasses abstratas *KeyPanel* (painéis de teclas) e *PaymentDevice* (dispositivos de pagamento).



Referências: Ian Sommerville, **Software Engineering**, 10ª ed., Pearson, 2016 (Cap. 5 — *System Modeling*). IAS — Institute of Industrial Automation and Software Engineering, Universität Stuttgart, **Exercises Software Engineering for Real-Time Systems**: Exercise 2 (Introduction to UML), Exercise 3 (Introduction to UML 2) e Exercise 8 (Use Case Analysis), WS 04/05.